

System Requirement Conflict Identification and Resolution using LLMs

Trung Kien Nguyen, Phong Tran, Quoc Co Luc
Swinburne University of Technology, Hawthorn, Victoria, Australia
Email: {104053642, 104334842, 103830572}@student.swin.edu.au

Abstract—System requirements often emerge from different stakeholders with diverse goals, priorities, challenges, motivations—leading to inconsistencies and conflicts that slow down system design. This project presents an automated framework for identifying and resolving conflicts using Large Language Models (LLMs). By utilizing the natural language understanding and reasoning capabilities of LLMs, the system extracts detailed requirements and usage contexts from multiple perspectives, detects contradictions across them, and proposes resolutions through guided strategies. The pipeline supports both functional and non-functional requirements and incorporates structured reasoning, persona-based simulation, and conflict identification analysis. As no existing solution offers fully automated conflict identification and resolution in requirements engineering, this research demonstrates a significant advancement toward intelligent, scalable, and inclusive software specification.

Index Terms—Requirements Engineering, Requirements Clustering, Conflict Identification and Resolution,

I. INTRODUCTION

In requirements engineering, one of the most persistent and complex challenges lies in reconciling conflicting needs and expectations from diverse stakeholders. As modern systems increasingly cater to heterogeneous user bases, it becomes critical to ensure that system requirements represent each user’s perspective accurately, while avoiding internal inconsistencies and contradictions. This report investigates an automated framework for detecting and resolving such requirement conflicts—particularly at the user story level, using the capabilities of the State-of-the-art (SOTA) Large Language Models (LLMs).

Early iterations of our pipeline, in the context of ALFRED system [1] - a virtual health assistant platform, revealed a major limitation in using LLMs (like OpenAI’s GPT models) for conflict identification and resolution. The model frequently misclassified the relationship between user stories—often labeling two unrelated or even supportive user stories as “conflict,” simply due to superficial differences in wording or hallucination because of very long prompt(s). This issue, raised during Weeks 6 and 7 of client consultations, underscored the opacity of LLM reasoning and the risk of overinterpreting vague language patterns as conflicts. It highlighted the need for a more structured

conflict detection approach(es) that could classify between true conflicts/inconsistencies and the non-conflicting ones.

This report aims to answer the following research questions:

- **RQ1:** How can we generate user stories that are specific, unambiguous, highlighting the persona’s singularities?
- **RQ2:** How can we reliably identify subtle or indirect requirement conflicts between user stories—especially when there is no obvious contradiction?
- **RQ3:** What resolution strategies are most suitable for addressing different types of conflicts identified between user stories?

Ultimately, this study contributes a methodology for automated system requirement conflict identification and resolution using LLMs, based on a case study in a realistic system (ALFRED). Furthermore, the pipeline is extensible and applicable to other systems where persona-driven requirement modeling is essential.

Structure. Section II provides the background of our study.

II. BACKGROUND

A. Contextualizing the Example System: ALFRED

The ALFRED system is a virtual assistant platform targeting older adults and their caregivers, medical staff, and developers. It embodies a complex socio-technical system with diverse user groups and multifaceted requirements spanning physical, cognitive, and social domains [1].

Its requirements address functional capabilities such as interaction assistance and personalized care, alongside nonfunctional concerns like privacy and accessibility. The system’s diversity necessitates sophisticated requirement engineering techniques capable of managing conflicting needs and priorities across stakeholder groups.

This study leverages ALFRED’s comprehensive user personas, use case, and user story documentation as an example and guidelines for the automated requirement generation, conflict identification, and

resolution, demonstrating applicability to similarly complex systems.

B. Related Works / Literature Review

User stories have become a fundamental artifact in requirements engineering, serving as concise, user-centric descriptions of desired system outcomes. They emphasize the end goal from the perspective of software users, facilitating collaboration among diverse stakeholder groups and encouraging creative problem-solving [1]. Deriving user stories from personas and use cases ensures that system functionalities reflect real user needs and contexts.

Requirements broadly divide into functional and non-functional types. Functional requirements specify concrete system behaviors, while non-functional requirements express quality attributes such as security, performance, and usability. The ALFRED system organizes requirements into six pillars, including four functional pillars, a general requirements pillar, and a developer core pillar [1].

Conflict identification and classification among requirements is a well-studied challenge in software engineering. Aldekhail et al. [2] surveyed eight prominent conflict detection techniques and assessed their suitability for various requirement types and engineering pipelines. Table I summarizes key approaches relevant to this study.

C. Foundational Concepts

a) User Stories and Personas: As mentioned above, user stories represent specific, goal-oriented requirements articulated from a user’s perspective. They are typically concise and non-technical, enabling cross-functional teams to understand user needs. Personas embody archetypal users with defined characteristics, goals, and challenges, serving as anchors for generating relevant user stories.

b) Functional vs. Non-Functional Requirements: Functional requirements describe what the system should do, while non-functional requirements capture how the system should behave, encompassing aspects like performance, security, and usability. Both are essential to system quality but often differ in clarity and conflict types.

c) Requirement Conflicts: Conflicts arise when two requirements (user stories) impose incompatible constraints or expectations. They can be categorized into mutually exclusive conflicts, partial conflicts, and semantic conflicts. Detecting and resolving these conflicts early is crucial to avoid design flaws and implementation issues.

III. METHODOLOGY

Fig. 1 provides an overview of our approach.

TABLE I
SUMMARY OF CONFLICT CLASSIFICATION TECHNIQUES
(LISTED IN [2])

Technique	Summary	Suitable for
Poort and de With’s technique [3]	Groups functional requirements based on non-functional requirements. Distinguishes conflicts as grouping or in-group conflicts.	Functional requirement classification (used in clustering)
Sadana and Liu [4]	Analyzes conflicts within non-functional requirements, defining mutually exclusive and partial conflicts.	Non-functional requirements
Heng and Ming	Uses multi-coordinated views to classify incomplete, overlapping, or disjoint requirements.	Functional and non-functional requirements
Butt et al.	Classifies conflicts based on requirement importance categories: mandatory, essential, optional.	Functional and non-functional requirements
Kim et al.	Proposes comprehensive classification covering both requirement types.	Functional and non-functional requirements
Moser et al.	Automates detection of complex semantic conflicts between requirements.	Functional and non-functional requirements
Chentouf [5]	Focuses on detecting conflicts in both functional and non-functional requirements using feature interaction analysis.	Functional and non-functional requirements
Mairiza and Zowghi	Specializes in non-functional requirement conflict classification.	Non-functional requirements

A. Overall Pipeline

The methodology follows a structured, iterative pipeline to generate comprehensive and consistent system requirements driven by user personas and use cases. The process is divided into the following key phases:

- **Phase 1:** Load detailed user personas and system-specific contexts capturing environment, constraints, and stakeholder perspectives to ground requirement generation and analysis.
- **Phase 2:** Use case generation and enrichment:
 - 2a: Randomly create skeleton use cases based on predefined rules.
 - 2b: Generate raw use cases by adding names and descriptions using LLMs.
 - 2c: Enrich use cases with life-like scenarios via LLMs.
 - 2d: Extract persona-specific tasks from scenarios using LLMs.
 - 2e: Deduplicate tasks within each persona across use cases.
- **Phase 3:** User story generation and classification:
 - 3a: Extract skeleton user stories from tasks

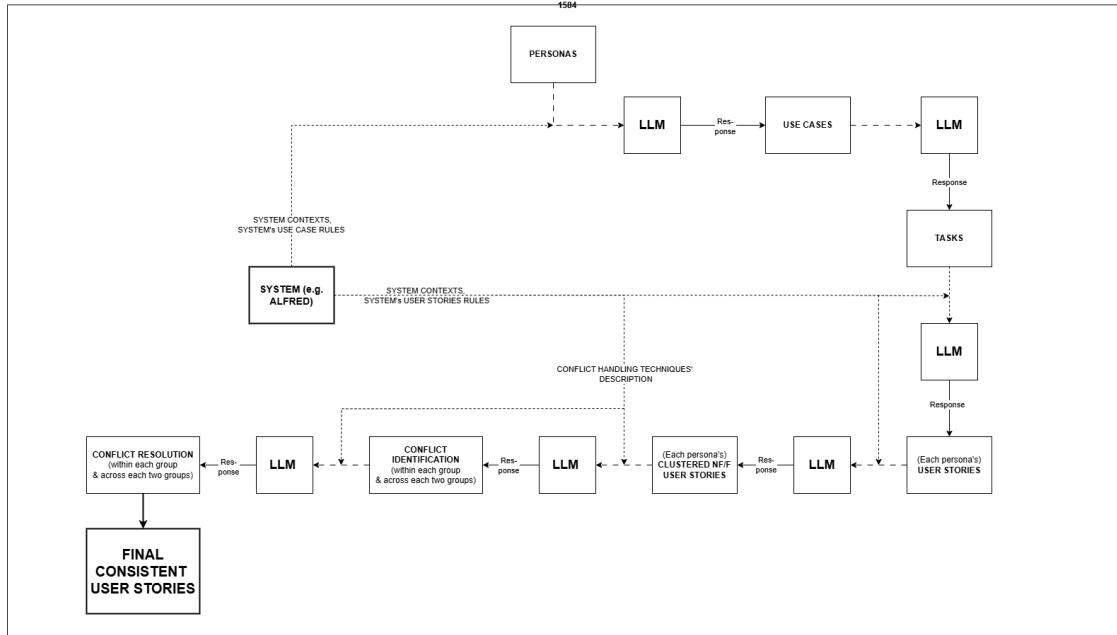


Fig. 1. Pipeline Overview - Summary version

- with associated use case and persona meta-data.
- 3b: Generate complete user story attributes (title, summary, priority) using LLMs.
- 3b-1: Verify that user story summaries reflect persona-specific perspectives.
- 3c: Classify each user story as functional or non-functional.
- 3d: Cluster non-functional user stories based on system pillars.
- 3e: Cluster functional user stories using their relationships with non-functional clusters.
- 3f: For each cluster, deduplicate the user stories made by each persona
- **Phase 4:** Non-functional user story conflict handling within groups:
 - 4a: Decompose non-functional user stories into atomic components.
 - 4b: Identify conflicts among personas within each group.
 - 4b-1: Verify identified conflicts to minimize hallucination risks.
 - 4c: Resolve conflicts within groups.
- **Phase 5:** Non-functional user story conflict handling across groups:
 - 5a: Identify conflicts across persona groups.
 - 5a-1: Verify these cross-group conflicts.
 - 5b: Resolve conflicts across groups.
- **Phase 6:** Functional user story conflict handling within groups:
 - 6a: Identify conflicts among personas within groups.
 - 6a-1: Verify these conflicts.
 - 6b: Resolve conflicts within groups.

- **Phase 7:** Functional user story conflict handling across groups:

- 7a: Identify conflicts between personas across groups.
- 7a-1: Verify identified conflicts.
- 7b: Resolve cross-group conflicts.

This modular and iterative pipeline facilitates continuous refinement as new personas, use cases, or system constraints emerge. Integrating automated LLM-based generation and conflict resolution accelerates the requirements engineering process while maintaining high quality and stakeholder relevance.

B. Data in This Project

Fig. 2 illustrates the initial data inputs, including system guidelines and persona groupings. The data foundations include:

a) *System-Related Guidelines:* These consist of structured, high-level system information and formalized rules guiding use case and user story generation. For ALFRED, the key system-related guidelines used are:

- **System Summary:** “ALFRED is a virtual assistant platform designed to support older adults in living independently, maintaining their physical and cognitive health, staying socially connected, and receiving effective and personalized care.”
- **Pillars:** The system requirements are organized into six pillars, briefly summarized here (full descriptions are provided in Appendix ??):
 - **Pi-DC: Developer Core** Foundational capabilities supporting external developers and ecosystem maintenance, including app marketplace management, API provision, developer support, and quality assurance.

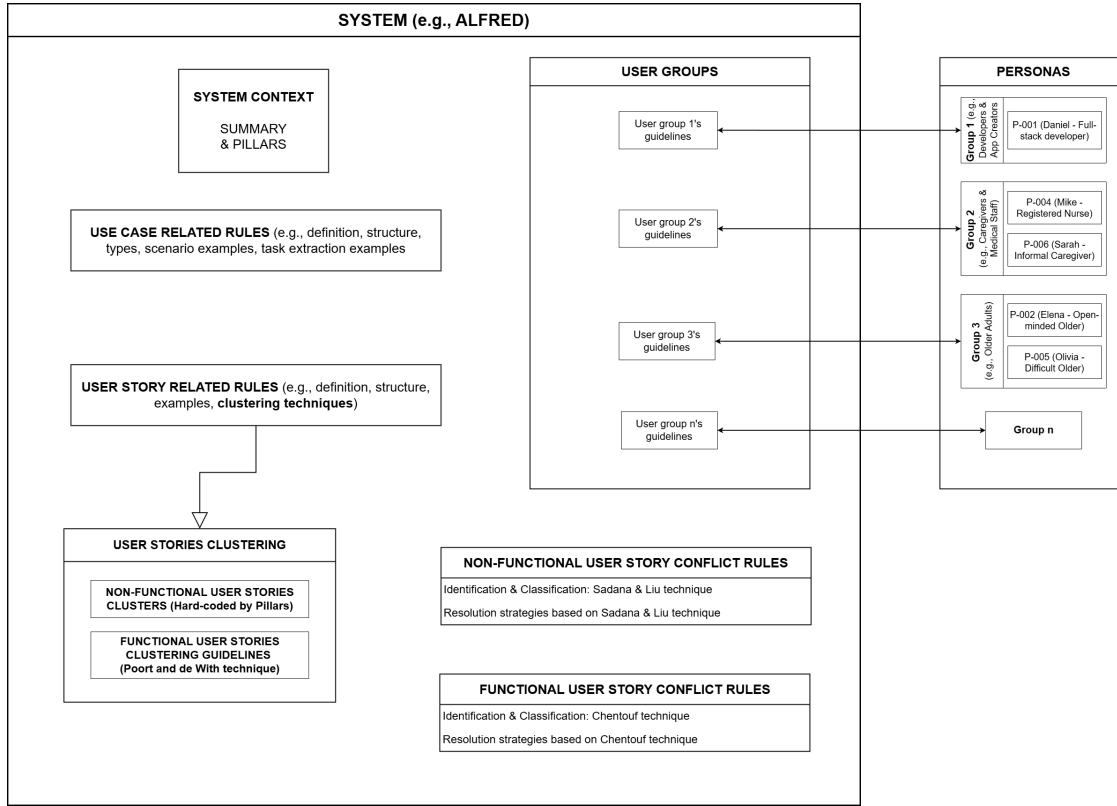


Fig. 2. Overview of Initial Data Inputs, including system guidelines and persona groupings.

- **Pi-GR: General Requirements** Cross-cutting system-wide capabilities essential for usability, safety, and extensibility, such as accessibility, data privacy, personalized interfaces, and device integration.
 - **Pi-001: Pillar 1 - User-Driven Interaction Assistant** Enables natural, voice-based user interactions for accomplishing daily tasks.
 - **Pi-002: Pillar 2 - Personalized Social Inclusion** Fosters social engagement and communication to reduce isolation.
 - **Pi-003: Pillar 3 - Effective & Personalized Care** Integrates health monitoring and data access for caregivers and medical staff.
 - **Pi-004: Pillar 4 - Physical & Cognitive Impairments Prevention** Promotes physical and cognitive stimulation through interactive activities.
- b) *User Personas and Stakeholder Groups:* The project incorporates five detailed personas representing three user groups:
- **Developers & App Creators:** *Daniel Chen (P-001)* — a conservative full-stack developer focused on accessible, secure healthcare apps.
 - **Caregivers & Medical Staff:** *Mike Johnson (P-004)* — a registered nurse with clinical workflow focus; *Sarah Thompson (P-006)* — an informal caregiver managing multiple elderly clients.
 - **Older Adults:** *Elena Rivera (P-002)* — tech-

curious with mild mobility issues; *Olivia (P-005)* — retired social worker with data privacy concerns.

Each persona includes unique attributes, goals, challenges, and singularities, detailed in Appendix VI-A.

c) *Use Case Types:* The project uses a predefined set of use case types to guide LLM-driven generation phases. These types emphasize personas' unique goals and challenges and include:

- Onboarding & Initial Setup
- Data Governance & Consent Enforcement
- Notification Preferences & Behavioral Nudging
- Simulation & Gamified Therapy
- Communication Style & Interpersonal Boundaries
- Sensor Usage & Wearable Compliance
- User Autonomy vs. System Control
- Check-in and Availability Negotiation
- Emotional Layering in Care Scenarios
- Health Data Sharing & Monitoring Boundaries
- Trust & Privacy Negotiation
- Developer Tools / Integration
- Marketplace Discovery

d) *User Story Guidelines:* User stories follow a defined structure including:

- Unique ID (e.g., US-XXX)
- Type: Functional or Non-functional
- Cluster: Thematic group based on system pillars

TABLE II
SUMMARY OF USER PERSONAS

Persona ID	User Group	Unique Features Summary
P-001: Daniel Chen	Developers & App Creators	Conservative full-stack developer focused on security, accessibility, and strict workflow enforcement in healthcare applications. Prefers modular backend and proactive user guidance.
P-002: Elena Rivera	Older Adults	Tech-curious, active older adult with mild mobility issues; open to automated health monitoring and social engagement; prefers spontaneous, informal communication style.
P-004: Mike Johnson	Caregivers & Medical Staff	Registered nurse with focus on efficient clinical workflows, patient privacy overridden by clinical needs, avoids non-emergency distractions, prefers minimal emotional interface.
P-005: Olivia	Older Adults	Retired social worker with strong privacy concerns, pessimistic about technology, prefers formal communication and minimal notifications; reluctant to wear health sensors.
P-006: Sarah Thompson	Caregivers & Medical Staff	Informal caregiver managing multiple clients; emotionally engaged, multitasking, supports interactive and bonding tools with patients.

- Title and Summary: Concise, persona-dominant descriptions emphasizing specific goals
- Persona and User Group associations
- Related Use Case
- Priority (1 - very high to 5 - out of scope)
- Raw task traceability

The LLM-generated content is controlled for language proficiency (B1 level English), prioritizing clarity, simplicity, and persona-centric perspectives, even when contradictory to the broader system context [1].

C. Use Case Task Extraction (Phase 2d) and Deduplication (Phase 2e)

In Phase 2d, the system extracts detailed persona-specific tasks from each generated use case scenario. The task extraction process is guided by a carefully constructed prompt fed into the Large Language Model (LLM). The prompt includes:

- The overall **system context**, providing background on the ALFRED platform and its goals.
- Information about all **personas involved** in the use case, including their unique attributes and goals.
- The full **use case scenario** narrative describing the interactions and flow.
- Instructions directing the LLM to extract a **diverse and realistic set of tasks** for each persona, covering both functional actions and non-functional expectations, while reflecting the persona's specific characteristics.

Phase 2e focuses on **task deduplication** within each persona's aggregated task list across all use cases. The LLM receives a prompt comprising:

- The **system context**, ensuring the tasks are interpreted within the correct operational environment.
- Detailed **information about each involved persona**.

- The full set of **extracted tasks per persona** before deduplication.
- Instructions asking the LLM to identify and flag **duplicate or semantically overlapping tasks** for removal, to maintain a clean and non-redundant task list.

This structured prompt engineering enables automated, high-quality extraction and cleaning of persona tasks, providing a precise foundation for subsequent user story generation and analysis.

D. User Story Generation

Fig. 3 shows the detailed pipeline for user story generation and clustering. User story generation is a multi-phase process transforming persona tasks (extracted from the use cases) into rich, persona-driven user stories:

- **Phases 3a & 3b:** For each deduplicated task per persona, a prompt is constructed including system context, user story guidelines (definitions, structure, examples), persona and user group information, the related use case, and prior user story summaries to avoid duplicates. The LLM generates a complete user story with title, summary, priority, and pillar.
- **Phase 3b-2:** Each generated user story summary is verified to ensure persona dominance. If verification fails, the story is updated using a prompt containing system context, guidelines, persona data, and the original user story.
- **Phase 3c:** User stories are classified as functional or non-functional by prompting the LLM with the system context, user story guidelines, and the story summary.

E. User Story Clustering

User story clustering is a critical step in organizing and analyzing the large volume of generated user stories. It facilitates systematic identification of thematic groups, which supports conflict detection, prioritization, and further refinement.

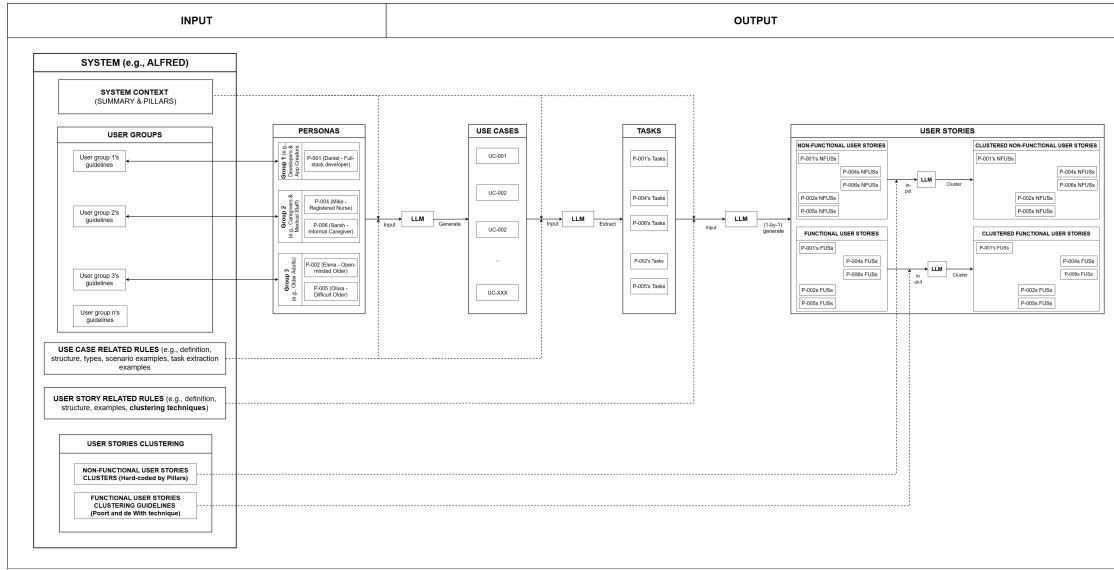


Fig. 3. Detailed pipeline for User Story Generation and Clustering, illustrating inputs, LLM-based generation steps, and clustering processes.

a) Phase 3d: Non-functional User Stories Clustering: Initially, non-functional requirements (user stories) are clustered using a hard-coded taxonomy inspired by established research in requirements engineering, notably the comprehensive review by Binkhonain and Zhao [6]. Since their taxonomy is designed for general software systems, it requires adaptation to the specific context of the ALFRED system.

Based on insights from the literature and the team's domain expertise, the following broad clusters were established for general software systems:

- **Security-related clusters (8):** Security, Integrity, Confidentiality, Authentication & Authorization, Access Control, Privacy, Cryptography/Encryption, Accountability.
- **Performance-related clusters (14):** Performance, Availability, Reliability, Scalability, Efficiency, Time behavior, Resource utilization, Maintainability, Portability, Supportability, Accuracy, Fault tolerance, Lifecycle, Deployability.
- **Usability-related clusters (8):** Usability, Look and Feel, Operability, System Interface, Accessibility, Understandability, Attractiveness, Flexibility.
- **Other/Mixed categories (7):** Functionality, Legal & Licensing, Interoperability, Modifiability, Testability, Traceability, Verifiability.

Following this initial clustering rationale, the team leveraged Large Language Models (LLMs) to rescope, update, specify, and prune the clusters, tailoring them to the ALFRED system's unique pillars. The refined non-functional user story clusters for ALFRED are:

- **General Requirements Clusters (2):**
 - *Security, Privacy & Reliability:* Consolidates system-wide requirements for secure data handling, privacy protection, reliable system behavior, high availability, failure resilience, and consistent performance.
 - *Accessibility & Physical Usability:* Encompasses physical and cognitive accessibility requirements alongside ergonomic considerations, including adaptive interfaces and hardware usability for users with impairments.
- **Pillar I: User-Driven Interaction Assistant (1):**
 - Encompasses non-functional expectations for voice-based controls, personalized interfaces, multimodal interactions, and adaptive UI support, ensuring usability for diverse user capabilities.
- **Pillar II: Personalized Social Inclusion (1):**
 - Focuses on requirements facilitating social engagement, event recommendations, and communication support to mitigate social isolation.
- **Pillar III: Effective & Personalized Care (1):**
 - Includes requirements related to health monitoring integration, data access for caregivers, and support for timely interventions.
- **Pillar IV: Physical & Cognitive Impairments Prevention (1):**
 - Covers stimulation through games, challenges, and guided activities tailored to individual physical and cognitive abilities.
- **Developer Core Clusters (2):**

- *API Integration & Development Support*: Developer tools, stable APIs, documentation, testing, and debugging support.
- *Marketplace & Interface Experience*: Infrastructure and UI design for the ALFRED marketplace, including app submission, discoverability, and minimal caregiver interfaces.

b) Phase 3e: Functional User Stories Clustering: Functional user stories are clustered based on their semantic relationships to the non-functional user stories, leveraging the approach proposed by Poort and de With [3]. In this method, each non-functional user story serves as a thematic anchor or cluster label. All functional user stories across different personas that are relevant to or support the objectives expressed by a given non-functional user story are grouped into the corresponding cluster.

This clustering strategy recognizes that functional requirements often realize or operationalize the qualities specified by non-functional requirements. By organizing functional stories around these quality anchors, the method facilitates a coherent structure for conflict identification and impact analysis, ensuring that functional implementations align with quality expectations.

The clustering process involves two main steps:

- 1) **Determining the Number of Functional Clusters**: The number of functional user story clusters is calculated proportionally based on the number of functional user stories relative to non-functional ones, according to the formula:
$$\text{Num of FUS Clusters} = \frac{\text{Num of FUSs}}{\text{Num of NFUSs}} \times \text{Num of NFUS Clusters}$$
 This ensures that clustering granularity scales appropriately with the volume of functional requirements.
- 2) **Cluster Generation (Phase 3e-1) and Assignment (Phase 3e-2)**: Using this calculated target number, a Large Language Model (LLM) is prompted with the complete set of non-functional user stories, system context, user story guidelines, and a description of the clustering technique to generate a list of meaningful cluster names for the functional stories. Subsequently, each functional user story is individually evaluated by the LLM, along with system context and the cluster list, to assign it to the most semantically appropriate cluster.

This approach supports nuanced grouping that reflects both the functional behaviors and their quality-driven context, enabling more effective downstream conflict detection, resolution, and system design prioritization.

F. Conflict Handling (Identification, Verification & Resolution)

This section details the methodology for identifying, verifying that identification, and resolving conflicts within the ALFRED system requirements, separated into non-functional and functional conflict handling.

- **Identifying**: A long prompt includes system context, technique applied description, and user story guidelines to identify whether the given user story pair includes conflicting or not.
- **Verifying the identification step**: Response made by a very long prompt often lead to LLM’s misleading or hallucination (which will be shown in the results below). Combining with a short prompt with only the pair of user stories and the system context, requiring a short answer of “Yes”/“No”, will be helpful to minimize this risk.
- **Resolution**: Write another long prompt to include the identified conflicting pair of user stories (verified), the system context and the technique description used, ask the LLM to resolve the conflict(s) and propose an updated version of consistent pair of the user stories.

1) Non-Functional User Story Conflict Handling: Non-functional requirements (NFRs) express quality attributes such as security, usability, and performance, which often conflict implicitly or explicitly. Following the framework proposed by Sadana and Liu [4], conflict detection in non-functional user stories leverages an integrated analysis of functional and non-functional requirements.

Their approach decomposes each non-functional user story (NFUS) into atomic, actionable components called lowest-level NFUSs. Conflicts are detected by pairwise comparison of these fragments. Two NFUSs conflict if they impose mutually exclusive expectations that cannot be satisfied simultaneously under the same conditions. Conflicts are classified as:

- **Mutually Exclusive Conflicts**: All fragments of one NFUS contradict all fragments of another, indicating fundamental incompatibility.
 - **A1 conflicts with B1, B2, B3**
 - or
 - **A2 conflicts with B1, B2, B3**
- **Partial Conflicts**: Only some fragments conflict, allowing for possible conditional resolutions or layered behaviors.
 - **A1 conflicts with B2 only, while it may not conflict with B1**
 - or
 - **A2 may or may not conflict with B1, B2, or B3**

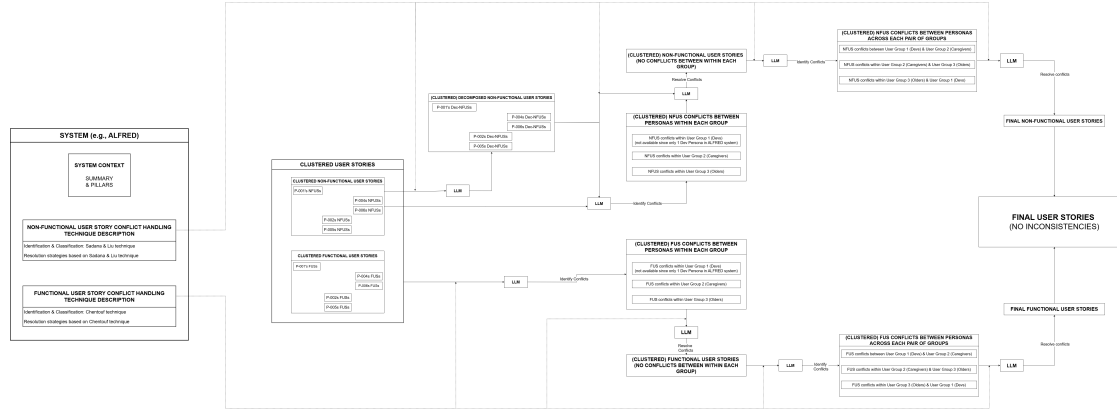


Fig. 4. Pipeline for Conflict Identification and Resolution illustrating detection methods for non-functional and functional user stories, and subsequent negotiation.

The conflict detection process is performed in:

- **Phase 4b:** Identification of non-functional user story conflicts from personas within each user group.
- **Phase 5a:** Identification of non-functional user story conflicts from personas across two groups.

Following each identification phase, the conflicts undergo:

- **Phase 4b-1:** Verification of identified non-functional user story conflicts within groups using concise prompts to minimize hallucination.
- **Phase 5a-1:** Verification of identified non-functional user story conflicts across groups similarly.

Resolution strategies involve contextual personalization, adaptive system behavior, or trade-off analysis to reconcile conflicting quality attributes, ensuring the final system meets stakeholder expectations without compromise to critical requirements:

- **Phase 4c:** Resolution of non-functional user story conflicts within groups.
- **Phase 5b:** Resolution of non-functional user story conflicts across groups.

2) Functional User Story Conflict Handling:

Functional requirements specify concrete system behaviors and are subject to direct logical contradictions. The conflict identification technique for functional user stories follows the method proposed by Chentouf et al. [5]. Two functional user stories conflict if they impose opposite expectations on system behavior that cannot be simultaneously satisfied.

The primary conflict types are:

- **Start-Forbid Conflict:** When one requirement mandates an operation on a triggering event, and another forbids the same operation under the same event.
- **Forbid-Stop Conflict:** When one requirement forbids an operation entirely, while another requires stopping it only conditionally.

- **Two Condition Events Conflict:** When conflicting conditions trigger the same operation inconsistently.
- **Two Operation Frequencies Conflict:** When requirements specify differing frequencies for the same operation.

These conflicts are detected by pairwise comparison of requirement attributes such as events, operations, agents, and conditions. The process is carried out in:

- **Phase 6a:** Identification of functional user story conflicts within each group.
- **Phase 7a:** Identification of functional user story conflicts across two groups.

Similar to non-functional conflicts, each identification phase is followed by verification:

- **Phase 6a-1:** Verification of identified functional conflicts within groups.
- **Phase 7a-1:** Verification of identified functional conflicts across groups.

Conflict resolution is conducted through:

- **Phase 6b:** Resolution of functional conflicts within groups.
- **Phase 7b:** Resolution of functional conflicts across groups.

a) Illustrative Examples::

- A start-forbid conflict might arise if one requirement states, “When the system starts, initialize logging,” while another states, “When the system starts, do not initialize logging.” Resolution may involve context-specific rules allowing logging only under certain modes.
- A two operation frequencies conflict occurs when one requirement calls for database backup every 5 minutes, and another every 15 minutes. A negotiated middle ground or configurable frequency resolves this.

b) Integrated Conflict Management Pipeline:

Both non-functional and functional conflict detection and resolution phases are embedded within the ALFRED requirements engineering pipeline, supported by structured representations, LLM-enhanced analysis, and hierarchical conflict refinement. Analysts iteratively verify and resolve conflicts using the system’s conflict hierarchy and recommended resolution patterns.

IV. EVALUATION (ALFRED CASE STUDY)

Because of the reasoning tasks in this project do not allow a free model with limited tokens to make the desired output, we test with only one cheap model, which is OpenAI’s gpt-4.1-mini.

We analysed 5 personas mentioned in the ?? and ??

A. Use Case Analysis

We analyzed the 15 use cases generated by the pipeline, detailed in Appendix Tables VIII, IX, X, and XI. The use cases cover a wide variety of types mentioned above (aligned with the personas’ distinct goals and challenges), reflecting ALFRED’s multi-faceted system objectives.

When it comes to the type of Use Case Type, only two type have the highest number and distribution percentage, with 2 use cases and 13%, respectively. The rest of all type are the same each other, with only 1 use case and 7%.

All user groups are well represented: Older Adults always engage in all use cases (100% coverage) with 15 times appearing, while Caregivers and Medical Staff takes the second place in participation with 11 time appearances (80% coverage). Developers comes last in approximately 67% coverage and 8 time appearances.

Persona-level involvement varies, P-005 (Older Adult) among the highest with 10 time appearances in every use cases with 67% coverage, then P-006 (Informal Caregiver) with P-002 (Difficult Older Adult) are the second highest with around 60% coverage and 9 appearances in all use cases. P-001 (Developer) and P-004 (Nurse) are among two of the last place in number of appearances with percentage of coverage, with respective 8, 7 times and 53%, 47%.

This distribution confirms the pipeline’s effectiveness in capturing relevant system interactions per persona and user group.

B. Task Extraction and Deduplication Results

Table XII in the Appendix summarizes task extraction and deduplication outcomes across personas. From 376 initially extracted tasks (all personas), 237

were identified as duplicates or semantically overlapping, leaving 139 unique, valid tasks (37% of the original).

Specifically, the no-need-to-deduplicate ratio, as shown in the table XII varied on every persona, with the highest around 78% (60 out of 77), in comparison with P-004 with only 6% (3/47), even though they are from the same Caregiver group. Then, P-001 (Developer) has the second highest unique rate, with below than the top just only 6% below (49 out of 70). The other two from Older group, which are P-002, P-005, stand at the second and third place respectively, with 16% and 14% (13/83 and 14/99). Overall, this suggests that Older Adults are most affected by the duplicated tasks problem when extracting them from use cases, followed by the Caregiver and Developer user groups

The table XIII shows the task distribution extracted from each use case. This suggests that the number of extracted tasks of each use case is quite balanced, supporting the accuracy of the predefined 13 Use case types rationale.

C. User story Generation

The table XIV show how the user stories are generated, and deduplicated. Note that the duplication check for User stories is executed after the clustering phases. Overall, there are about 1/3 user stories associated with all personas are detected as duplicated and removed.

However, the problem of similar information in user stories still exists dramatically and the team has not completely solved it, despite trying to change the prompt many times. The team has thought that the only way is that, instead of inputting a number of user stories (by cluster and persona) into LLM and letting it show duplicated ones as it is now in the pipeline, each time only input 2 user stories into the prompt, so the result will be as accurate as possible. However, we decided not to choose this option because of the huge limitation in Execution time (Max Time complexity = $O(n^2)$)

On the other hand, the table XV shows the distribution of the non-functional and functional user stories. The number of FUSs is generally twice the number of NFUSs. This ratio, as mentioned in the III, also affects how Functional user stories are clustered. Specifically, the number of clusters for Non-functional user stories is hard-coded at 8, and, with the application the Poort and de With’s technique [3], along with the team’s proposal, we define the number of clusters for functional user stories as:
$$\text{Num of FUS Clusters} = \frac{\text{Num of FUSs}}{\text{Num of NFUSs}} \times \text{Num of NFUS Clusters} = \frac{91}{48} \times 8 \approx 15$$

This ratio is to ensure that the number of clusters for Functional user stories is not too large (which is 48 if we merely apply Poort and de With technique

[3]). In fact, previously, the team has applied a large number of FUS clusters, resulting in there is only 1-2, or even 0 user stories in many clusters, hence no conflicts are found.

D. Conflict

As mentioned above, there are four cases to handle the conflicts, which are

- Non-functional conflicts within each group
- Functional conflicts within each group
- Non-functional conflicts across each pair of groups
- Functional conflicts across each pair of groups

The differences are the technique applied (Sadana and Liu [4] for Non-functional and Chentouf [5] for Functional) and the different between user groups. Basically all conflict handlings have to go through 3 phases using the LLM:

- Phase a. Conflict identifying with long-prompt (system context + technique description + user story guidelines + user group description + two user stories needing to identify)
- Phase b. Verify the identified conflict again with a short prompt (only system summary + two identified conflicting user stories)
- Phase c. Propose the update version of thee two conflicting user stories (Prompt: System context + technique description + two conflicting user stories)

The reason why we need a short-prompt verifying (Phase b) after identification (Phase a) is that a long and detailed prompt, in many cases, mislead and confuse the LLM, and decrease the reasoning ability of the LLM. We noticed this problem in Week 10 as one of the main reasons why we have experienced the identified conflicting user stories (with long prompts), which are actually irrelevant or supporting each other. Whereas, after simply putting those two user stories again to ChatGPT with only a short summary of the ALFRED (yes/no question), we get the surprising answer that they are not conflicting.

To support our findings, the tables XVIII, XIX, XX, XXI shows that the Confliting Verification Success rate (which mean both Phase a and Phase b state that the pair is in Conflict) is under 50% in total: $\frac{0+2+13+36}{0+16+16+88} = \frac{51}{120} \approx 42\%$

Moreover, to illustrate the effectiveness of the short-prompt technique, we have manually checked all identified and verified conflicting pairs. The result table of User Story Conflict Verification Analysis by Human includes 120 conflicting pairs of user stories identifying by Phase a, which are partially denied by the Verification Phase b. The team has manually check how much the Verification results (Conflict/Invalid conflict) would match the team's idea (Conflict/Not conflict). As a result, about $\frac{85}{120} \approx$

71% pairs of conflicting LLM verifications match the ideas of the team. The remaining 29% (35 pairs) are:

- mostly (33 conflicts) when the identified conflict is verified as a Valid Conflict as well, while the team does not think it is actually a clear conflict/inconsistency
- Only 2 conflicts (FCAT-041 and FCWI-016), which have been identified as conflicts in Phase a (long-prompt), but denied in Phase b (short-prompt verification), while the team thinks they are still the conflicts

When it comes to conflict resolution, the pipeline, after Verification with a short prompt, detects a total of 51 conflicts, which are then updated to remove inconsistencies in the user stories. The team has also manually decided whether we will agree with the resolution proposal for each conflicting pair (See table User story Conflict Resolution Analysis by Human). Including about 33 pairs that the team did not think were conflicts earlier, the team has also disagreed with 3 additional pairs in terms of resolution ideas, the rate of the team's agreement on resolution proposals is about: $100\% - (\frac{33+3}{51}) \approx 29\%$

If we do not count the 33 pairs that the team did not think were conflicts earlier, which means the checked pairs are $51 - 33 = 18$, then the team's agreement rate is $100\% - (\frac{3}{18}) \approx 83\%$

V. REFERENCES

- [1] A. Consortium, "D2.3 – User Stories and Requirement Analysis," ALFRED Project, Tech. Rep. Version 1.4, March 2022.
- [2] M. Aldekhail, A. Chikh, and D. Ziani, "Software requirements conflict identification: Review and recommendations," *International Journal of Advanced Computer Science and Applications (IJACSA)*, vol. 7, no. 10, pp. 325–335, 2016. [Online]. Available: https://thesai.org/Downloads/Volume7No10/Paper_43-Software_Requirements_Conflict_Identification.pdf
- [3] E. R. Poort and P. H. N. de With, "Resolving requirement conflicts through non-functional decomposition," in *Proceedings of the Fourth Working IEEE/IFIP Conference on Software Architecture (WICSA'04)*. IEEE, 2004, pp. 12–21. [Online]. Available: <https://ieeexplore.ieee.org/document/1310698>
- [4] V. Sadana and X. F. Liu, "Analysis of conflicts among non-functional requirements using integrated analysis of functional and non-functional requirements," *Proceedings of the 31st Annual International Computer Software and Applications Conference (COMPSAC 2007)*, pp. 31–38, 2007. [Online]. Available: <https://ieeexplore.ieee.org/document/4291007>
- [5] Z. Chentouf, "Detecting oam&p requirement conflicts using a feature interaction approach," in *International Journal of Network Management*, vol. 22, no. 2. Wiley Online Library, 2012, pp. 95–103. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S1319157814000056>
- [6] M. Binkhonain and L. Zhao, "A review of machine learning algorithms for identification and classification of non-functional requirements," *Expert Systems with Applications: X*, vol. 1, p. 100001, 2019. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S2590188519300010>

VI. APPENDIX

A. User Personas

TABLE III
PERSONA P-001 – DANIEL CHEN

ID	P-001
Name	Daniel Chen
Role	Full-stack developer creating healthcare applications
Tagline	I am a conservative developer, who build accessible, secure healthcare apps for elderly and caregivers.
Demographic Data	Age: 32; Marital Status: Single; Lives in rural area, works remotely; Bachelor of Computer Science; No children
Core Characteristics	Efficiency-Oriented, Conservative, Controlling other's data
Core Goals	Ensure technical enablement, enforce control of options, hard games only, always send reminders, restrict user freedom
Typical Challenges	Balancing modernity and compatibility, must push caregivers, backend routines, limited collaboration
Singularities	Forces notifications and data collection, modular backend, conservative interaction model
Main Actions	Push installs and reminders to users and caregivers forcibly
Expertise	Secure healthcare development, backend architecture, user data handling
Workplace	Remote work, home office with cloud tools

TABLE IV
PERSONA P-002 – ELENA RIVERA

ID	P-002
Name	Elena Rivera
Role	Tech-curious older adult with mild mobility issues
Tagline	I am an open minded and dependent person and I want to live a good life. It doesn't matter if the ALFRED system is controlling to ensure my health.
Demographic Data	Age: 60; Married; Urban resident; High school graduate; 2 children
Core Characteristics	Optimistic, Open-minded, Dependent, Flexible, Curious
Core Goals	Exercise suggestions, automatic activation, wearable sensors, spontaneous interaction
Typical Challenges	Navigation, socializing, tech learning curve, motivation drops
Singularities	Embraces wearables, hard challenges, all data shared freely
Main Actions	Accepts all notifications, video check-ins, wants ALFRED to teach usage, chats with strangers
Expertise	None listed
Workplace	None

TABLE V
PERSONA P-004 – MIKE JOHNSON

ID	P-004
Name	Mike Johnson
Role	Registered Nurse providing urban healthcare services
Tagline	I want to independently use digital tools to support my urban patients and explore system features at my own pace.
Demographic Data	Age: 43; Single; Metropolitan hospital nurse; Master's degree; No children
Core Characteristics	Task-focused, Workflow-driven, Time-constrained
Core Goals	Efficient care, skip emotional layers, immediate access to patient data
Typical Challenges	Tech transitions, time constraints, tiredness, skips exercise
Singularities	Rejects emotional design, only health data matters, forces sensors
Main Actions	Automatically speaks to patients, blocks non-medical functions, logs all actions
Expertise	Geriatric care, clinical monitoring, urban medical systems
Workplace	Urban hospitals

TABLE VI
PERSONA P-005 – OLIVIA

ID	P-005
Name	Olivia
Role	Retired social worker and community volunteer
Tagline	I am a difficult, pessimistic, and dependent person and I need help to monitor and improve my recovery constantly. But I often feel unsafe due to tech.
Demographic Data	Age: 75; Widow; Lives alone in suburb; Bachelor of Social Work; 1 child
Core Characteristics	Difficult, Pessimistic, Emotionally fragile, Independent
Core Goals	Disable almost all notifications, avoid data sharing, prefer manual control, no exercise
Typical Challenges	Privacy, health resistance, low tech comfort
Singularities	Blocks app installs, no sensors or chat, only official medical channels
Main Actions	Formal-only communication, push-to-talk only, offline check-ins preferred
Expertise	Community engagement (retired)
Workplace	None

TABLE VII
PERSONA P-006 – SARAH THOMPSON

ID	P-006
Name	Sarah Thompson
Role	Informal caregiver managing multiple elderly clients
Tagline	I need reliable and secure tools to monitor multiple clients, with a strong focus on privacy and clarity in shared environments.
Demographic Data	Age: 38; Married; Lives with family; Diploma in Health Science; 1 child
Core Characteristics	Emotionally engaged, Multitasking, Flexible working hours
Core Goals	Enable bonding, respect privacy, support informal care, always available
Typical Challenges	Overlap stress, social-emotional balance, context switching
Singularities	Plays games, welcomes informal bonding, manages care flow at home
Main Actions	Encourages ALFRED explanation, accepts all alerts, disables auto-logging
Expertise	Elder care, home routines, family-integrated care
Workplace	Patient homes, suburban and rural areas

B. Result analysis - Use case

TABLE VIII
SUMMARY OF GENERATED 15 USE CASES

Total number of Use cases: 15					
ID	Type	Personas	UserGroups	Pillars	Name
UC-001	User Autonomy vs. System Control	P-006, P-001, P-002, P-005, P-004	Caregivers and Medical Staff, Developers and App Creators, Older Adults	Developer Core, General Requirements, Pillar 1 - User-Driven Interaction Assistant	Balancing Autonomy and Control
UC-002	Communication Style & Interpersonal Boundaries	P-005	Older Adults	Pillar 1 - User-Driven Interaction Assistant, Pillar 2 - Personalized Social Inclusion	Formal Communication and Privacy Control
UC-003	Health Data Sharing & Monitoring Boundaries	P-002, P-006	Caregivers and Medical Staff, Older Adults	Developer Core, Pillar 3 - Effective & Personalized Care	Dynamic Health Data Sharing Control
UC-004	Notification Preferences & Behavioral Nudging	P-004, P-002	Caregivers and Medical Staff, Older Adults	General Requirements, Pillar 1 - User-Driven Interaction Assistant	Configurable Notification and Nudging
UC-005	Simulation & Gamified Therapy	P-006, P-002, P-005, P-001	Caregivers and Medical Staff, Developers and App Creators, Older Adults	Developer Core, Pillar 4 - Physical & Cognitive Impairments Prevention	Interactive Multi-User Simulation Therapy
UC-006	Trust & Privacy Negotiation	P-005, P-006, P-001, P-004	Caregivers and Medical Staff, Developers and App Creators, Older Adults	Developer Core, General Requirements, Pillar 1 - User-Driven Interaction Assistant	Personalized Privacy Consent Management
UC-007	Data Governance & Consent Enforcement	P-006, P-001, P-005	Caregivers and Medical Staff, Developers and App Creators, Older Adults	Developer Core, General Requirements	Multi-Party Consent Coordination
UC-008	Sensor Usage & Wearable Compliance	P-006, P-004, P-002, P-005	Caregivers and Medical Staff, Older Adults	Developer Core, Pillar 3 - Effective & Personalized Care	Wearable Sensor Compliance Management
UC-009	Emotional Layering in Care Scenarios	P-004, P-006, P-002, P-005	Caregivers and Medical Staff, Older Adults	General Requirements, Pillar 2 - Personalized Social Inclusion	Emotional Engagement in Care
UC-010	Marketplace Discovery	P-005, P-001	Developers and App Creators, Older Adults	Developer Core, General Requirements	Selective App Installation Control
UC-011	Developer Tools / Integration	P-001, P-005, P-004	Caregivers and Medical Staff, Developers and App Creators, Older Adults	Developer Core, General Requirements, Pillar 2 - Personalized Social Inclusion	Enforced Integration and Compliance
UC-012	Onboarding & Initial Setup	P-005, P-001	Developers and App Creators, Older Adults	General Requirements, Pillar 1 - User-Driven Interaction Assistant, Pillar 2 - Personalized Social Inclusion	Controlled Onboarding with Privacy Focus
UC-013	Emotional Layering in Care Scenarios	P-006, P-002	Caregivers and Medical Staff, Older Adults	Pillar 2 - Personalized Social Inclusion, Pillar 3 - Effective & Personalized Care	Contextual Emotional Support Coordination
UC-014	Check-in and Availability Negotiation	P-002	Older Adults	Pillar 1 - User-Driven Interaction Assistant, Pillar 3 - Effective & Personalized Care	Health Check-In and Availability
UC-015	Notification Preferences & Behavioral Nudging	P-004, P-002, P-001, P-006	Caregivers and Medical Staff, Developers and App Creators, Older Adults	Developer Core, General Requirements, Pillar 1 - User-Driven Interaction Assistant, Pillar 3 - Effective & Personalized Care	Adaptive Notification and Nudging

TABLE IX
USE CASE ANALYSIS BY PREDEFINED TYPES

Use Case Type	Number of Use Cases	Distribution Percentage
Notification Preferences & Behavioral Nudging	2	13%
Emotional Layering in Care Scenarios	2	13%
Health Data Sharing & Monitoring Boundaries	1	7%
Communication Style & Interpersonal Boundaries	1	7%
User Autonomy vs. System Control	1	7%
Trust & Privacy Negotiation	1	7%
Sensor Usage & Wearable Compliance	1	7%
Simulation & Gamified Therapy	1	7%
Marketplace Discovery	1	7%
Developer Tools / Integration	1	7%
Onboarding & Initial Setup	1	7%
Check-in and Availability Negotiation	1	7%
Data Governance & Consent Enforcement	1	7%
Total	15	100%

TABLE X
USE CASE COVERAGE ACROSS USER GROUPS

User Group	Number of Appearances	Coverage Percentage
Older Adults	15	100%
Caregivers and Medical Staff	11	73%
Developers and App Creators	8	53%

TABLE XI
USE CASE INVOLVEMENT PER PERSONA

Persona	Number of Appearances in All Use Cases	Coverage Percentage
P-005	10	67%
P-006	9	60%
P-002	9	60%
P-001	8	53%
P-004	7	47%

C. Task Analysis

TABLE XII
TASK EXTRACTION (PHASE 2D) AND DEDUPLICATION (PHASE 2E) ANALYSIS PER PERSONA

Persona	Total Number of Tasks Extracted (from use cases)	Number of Duplicated Tasks	Number of Unique Tasks	Percentage of Unique Tasks
P-001	70	21	49	70%
P-002	83	70	13	16%
P-004	47	44	3	6%
P-005	99	85	14	14%
P-006	77	17	60	78%
Total	376	237	139	37%

TABLE XIII
UNIQUE TASK DISTRIBUTION BY PERSONAS AND USE CASES

Use Case ID	P-001	P-002	P-004	P-005	P-006	Total
UC-001	5	2	0	1	5	13
UC-002	0	0	0	1	0	1
UC-003	0	2	0	0	6	8
UC-004	0	1	0	0	0	1
UC-005	7	1	0	1	6	15
UC-006	3	0	1	3	7	14
UC-007	8	0	0	0	8	16
UC-008	0	2	1	1	6	10
UC-009	0	0	0	2	8	10
UC-010	7	0	0	2	0	9
UC-011	7	0	1	1	0	9
UC-012	7	0	0	2	0	9
UC-013	0	2	0	0	9	11
UC-014	0	1	0	0	0	1
UC-015	5	2	0	0	5	12
Total	49	13	3	14	60	139

D. Result analysis - User story

TABLE XIV
USER STORY UNIQUENESS ANALYSIS BY PERSONAS

Persona	Total number of User Stories (from unique tasks)	Number of Duplicated User Stories	Number of Unique User Stories	Percentage of Unique User Stories
P-001	49	26	23	47%
P-002	13	2	11	85%
P-004	3	0	3	100%
P-005	14	3	11	79%
P-006	60	17	43	72%
Total	139	48	91	65%

TABLE XV
USER STORY UNIQUENESS ANALYSIS BY TYPES (FUNCTIONAL/NON-FUNCTIONAL

Type	Total number of User Stories (from unique tasks)	Number of Duplicated User Stories	Number of Unique User Stories	Percentage of Unique User Stories
Functional	99	29	70	71%
Non-Functional	40	19	21	52%
Total	139	48	91	65%

TABLE XVI
NON-FUNCTIONAL USER STORY CLUSTERING BY PERSONAS

Cluster	P-001	P-002	P-005	P-006	Total
API Integration & Development Support	2	0	0	0	2
Accessibility & Physical Usability	0	0	4	0	4
Effective & Personalized Care	0	0	1	0	1
Marketplace & Interface Experience	1	0	0	0	1
Personalized Social Inclusion	0	0	2	0	2
Security, Privacy & Reliability	5	0	2	3	10
User-Driven Interaction Assistant	0	1	0	0	1
Total	8	1	9	3	21

TABLE XVII
FUNCTIONAL USER STORY CLUSTERING BY PERSONAS

Cluster	P-001	P-002	P-004	P-005	P-006	Total
(Unclustered)	0	1	0	0	5	6
App Security	2	0	0	0	0	2
Backend Updates	6	0	0	0	0	6
Communication Style	0	0	0	0	2	2
Consent Enforcement	4	0	1	0	2	7
Consent Module Updates	0	0	0	0	1	1
Consent-Based Communication	0	0	0	1	5	6
Data Sharing Control	0	1	0	0	2	3
Feature Explanation	0	2	0	0	0	2
Manage care time across clients	0	0	0	0	1	1
Notification Control	1	2	1	1	12	17
Online Meetings	1	1	0	0	1	3
Reminder Control	0	1	0	0	3	4
Shared Device Privacy	0	0	0	0	3	3
Silent Mode	0	0	1	0	0	1
Social Nudges Control	0	1	0	0	3	4
Voice Activation Control	1	1	0	0	0	2
Total	15	10	3	2	40	70

E. Conflict Analysis

TABLE XVIII
COUNT OF NON-FUNCTIONAL USER STORY CONFLICTS WITHIN EACH USER GROUP

User Group Pair	Total Number of Identified Conflicts	Verified	Failed Verification	Conflict Verification Success Rate
Total	0	0	0	0%

TABLE XIX
COUNT OF FUNCTIONAL USER STORY CONFLICTS WITHIN EACH USER GROUP

User Group	Total Number of Identified Conflicts	Verified	Failed Verification	Conflict Verification Success Rate
Caregivers and Medical Staff	14	2	12	14%
Developers and App Creators	0	0	0	0%
Older Adults	2	0	2	0%
Total	16	2	14	12%

TABLE XX
COUNT OF NON-FUNCTIONAL USER STORY CONFLICTS ACROSS EACH PAIR OF USER GROUPS

User Group Pair	Total Number of Identified Conflicts	Verified	Failed Verification	Conflict Verification Success Rate
Caregivers and Medical Staff vs Developers and App Creators	8	6	2	75%
Caregivers and Medical Staff vs Older Adults	3	3	0	100%
Developers and App Creators vs Older Adults	5	4	1	80%
Total	16	13	3	81%

TABLE XXI
COUNT OF FUNCTIONAL USER STORY CONFLICTS ACROSS EACH PAIR OF USER GROUPS

User Group Pair	Total Number of Identified Conflicts	Verified	Failed Verification	Conflict Verification Success Rate
Caregivers and Medical Staff vs Developers and App Creators	26	25	1	96%
Caregivers and Medical Staff vs Older Adults	57	8	49	14%
Developers and App Creators vs Older Adults	5	3	2	60%
Total	88	36	52	41%